

SIMATS ENGINEERING

Department of Computer Science and Engineering

Course Code:	CSA1512	Assessment:	CO5 AT1
Course Name:	Cloud Computing and Big Data Analytics	Type:	Constraint-Based Problem
CO Statement:	Apply big data storage, integration, Hadoop, HDFS, and MapReduce concepts for distributed data processing		

Part A: HDFS Fundamentals – Lesson Notes

What is HDFS?

The Hadoop Distributed File System (HDFS) is a distributed storage system designed to reliably store very large files (hundreds of MB or more) across multiple machines, to stream those files at high bandwidth, and to tolerate node failures gracefully. Key design principles:

1. Fault tolerance through replication – files are replicated across multiple nodes.
2. Write-once-read-many access – simplifies data consistency.
3. High throughput over latency – optimized for batch processing, not real-time.

Lesson Note: HDFS vs. Traditional File Systems

HDFS is NOT a general-purpose file system like ext4 or NTFS. It trades random access and POSIX compliance for high throughput on large files and fault tolerance. A single HDFS file might be 100GB+, split into blocks across dozens of DataNodes.

HDFS Architecture

An HDFS cluster consists of three main components:

Component	Role	Count per Cluster
NameNode	Manages file system namespace; maintains file system tree + metadata for files/folders. Does NOT store actual data.	1 (or 1 primary + 1 secondary for HA)
Secondary NameNode	NOT a backup NameNode. Performs housekeeping (merges namespace image + edit logs). Prevents NameNode's edit logs from becoming too large.	1
DataNodes	Store the actual data blocks. Perform block creation, deletion, replication on instruction from NameNode.	Many (typically 10–100s)

Critical Insight: Single Point of Failure

The NameNode holds the entire file system namespace in RAM. If it crashes, the cluster is unusable until restart. This is why a Secondary NameNode (or HA pair) is essential in production.

Blocks & Replication

HDFS breaks each file into blocks (default 128MB or 256MB). Each block is replicated across multiple DataNodes. The replication factor (RF) controls how many copies exist.

Replication Factor	Fault Tolerance	Storage Overhead	Typical Use Case
1	None – single node failure loses data	100%	Non-critical data, dev/test
2	Survive 1 rack failure (if nodes on same rack); single node failure OK	200%	Development clusters
3	Survive 1 rack failure + 1 node in another rack	300%	Production (Hadoop default)

Rack Awareness Strategy
With RF=3: place 2 replicas on same rack (fast write); 1 replica on different rack (survival of rack-level disaster). This balances write speed (same rack) vs. fault tolerance (different racks).

Block Sizing & Data Locality

Block size (default 128MB) affects:

- Larger blocks → fewer blocks → fewer map tasks. May reduce parallelism.
- Smaller blocks → more blocks → more map tasks → better parallelism but more NameNode pressure.

Data locality principle: MapReduce scheduler tries to place mappers on nodes holding the input block. Three tiers:

1. Node-local (best): mapper runs on same node as block.
2. Rack-local (ok): mapper on same rack as block, different node.
3. Off-rack (worst): mapper elsewhere, data moves over network.

Part B: Main Scenario – Web Log Analysis Cluster

Problem Statement

Scenario: Medi Queue AI is deployed across 100 hospital/clinic branches. Each branch's queue-management service generates ~5GB per day of queue and appointment event logs (patient check-ins, token/queue-position generation, wait-time updates, cancellations and no-shows, and notification-delivery logs).

- Store 2 years of logs (36.5TB raw data)
- Run daily MapReduce jobs to analyze queue behaviour and no-show patterns across branches
- Survive single rack failure without data loss
- Minimize hardware costs while ensuring high availability

Lesson Note: Constraint-Based Design
Real-world systems have conflicting constraints: high availability wants more replication (higher cost); cost minimization wants less replication. Your job is to balance them under business constraints.

Step 1: Calculate Storage Requirements

Instructions: Fill in the worksheet below.

Metric	Value	Your Answer
--------	-------	-------------

Raw data (100 servers × 5GB/day × 365 × 2 years)	36,500 GB = 36.5 TB	36.5 TB (confirmed — 100 branches × 5GB/day × 730 days)
Replication factor assumed	3× (survive 1 rack fail)	3× (production default — required to survive a rack failure)
Total raw disk needed (raw data × RF)		36.5 TB × 3 = 109.5 TB
Overhead for metadata (~5% of raw)		109.5 TB × 0.05 = 5.475 ≈ 5.5 TB
Usable capacity after overhead		109.5 + 5.5 = 115 TB

Guided Calculation

Total raw disk = 36.5 TB × 3 = 109.5 TB

Metadata overhead ≈ 109.5 × 0.05 = 5.5 TB

Usable capacity ≈ 109.5 + 5.5 = 115 TB

Step 2: Size the DataNode Hardware

Typical DataNode specs: 12TB disk, 64GB RAM. Design the cluster:

Item	Calculation	Your Answer
Number of DataNodes needed	115 TB ÷ 12 TB/node = ~10 nodes	115 TB ÷ 12 TB/node = 9.6 → round up to 10 DataNodes
Total RAM across cluster	10 nodes × 64 GB = 640 GB	10 nodes × 64 GB = 640 GB total cluster RAM
NameNode heap size (rule: 1GB per 1M files)	If ~50M files → 50GB NameNode heap	Medi Queue AI's logs are naturally low file-count (one rolling log stream per branch, not one file per queue event): ~100 branches × 730 days ≈ 73,000 log files if rotated daily, or far fewer if rotated weekly/monthly. At 1GB heap per 1M files, 73,000 files needs well under 1GB of NameNode heap — a default NameNode heap (e.g. 4–8GB) is more than sufficient. This is a deliberate design choice: batching queue events into large rolling files (not one file per check-in/token event) avoids the HDFS small-file problem entirely.

Step 3: Replication & Rack Topology

Design the rack layout. Assume 2 racks (each can hold ~5 nodes).

Topology Layout

Rack 1: 5 DataNodes (DN1–DN5) Rack 2: 5 DataNodes (DN6–DN10) NameNode + Secondary NameNode: Separate machine (NOT on a DataNode).

Fill in the replication strategy:

Replica #	Placement	Rationale
-----------	-----------	-----------

1st replica	Local node in Rack 1 (the DataNode the writer/client is closest to, e.g. DN1)	Fastest possible write — no network hop needed for the first copy.
2nd replica	A different DataNode in the same Rack 1 (e.g. DN2)	Still a fast, same-rack (top-of-rack switch) write, giving an immediate second copy with minimal latency penalty.
3rd replica	A DataNode in Rack 2 (e.g. DN6)	Cross-rack copy — the only replica that survives a full Rack 1 failure, meeting the “survive single rack failure” requirement.

Step 4: Fault Tolerance Analysis

Scenario: Rack 1 fails (all 5 DataNodes go down).

Instructions: Analyze data loss for different replica placements.

Expected Insight

If all 3 replicas are in Rack 1, losing Rack 1 means total data loss. The standard strategy spreads replicas: 2 in one rack, 1 in another, preventing rack-level loss.

Your Turn:

1. If we place replicas as: 2 in Rack 1, 1 in Rack 2, what happens when Rack 1 fails?

If replicas are placed as 2 in Rack 1 and 1 in Rack 2, and Rack 1 fails completely, the 2 replicas on Rack 1 are lost but the 1 replica in Rack 2 survives — no queue/appointment data is permanently lost. However, every affected block is now under-replicated (down from RF=3 to RF=1), so patient queue history is briefly running with zero redundancy until the NameNode detects the under-replication and instructs DataNodes in Rack 2 (and any recovered capacity) to re-replicate those blocks back up to RF=3. During that re-replication window the system is exposed to a second failure, so this window should be minimized with fast inter-rack bandwidth and prompt monitoring/alerting.

Step 5: MapReduce Job Scheduling

A daily MapReduce job processes the logs: input = 5GB/day × 2 years = 36.5TB.

Metric	Calculation	Your Answer
Block size	128 MB (default)	128 MB (kept at default — balances parallelism against NameNode load for this data size)
Number of blocks	$36.5 \text{ TB} \div 128 \text{ MB} = \sim 292,000 \text{ blocks}$	$36.5 \text{ TB} \div 128 \text{ MB} \approx 292,000 \text{ blocks}$
Number of mappers (≈ 1 per block)	$\sim 292,000$ mappers	$\approx 292,000$ mappers for a full historical reprocessing run (the daily incremental job only touches $\sim 500\text{GB/day} \approx 4,000$ new blocks/mappers)
Data locality %	If 3 replicas well-distributed, expect 90%+ node-local	With the Step 3 rack-aware placement (2 copies in the writer's rack, 1 cross-rack), expect roughly 90%+ node-local mapper placement, since the scheduler can almost always find a replica on or near the requested node

Step 6: Cost-Benefit Analysis

Calculate total cost. Assume: \$500 per DataNode (hardware + network), \$1000 annual per DataNode (power, cooling, ops).

Cost Item	Calculation	Total
Initial hardware (10 DataNodes)	$10 \times \$500$	$10 \times \$500 = \$5,000$
Annual operations (10 DataNodes)	$10 \times \$1000$	$10 \times \$1,000 = \$10,000/\text{year}$
2-year total cost	$(\text{Hardware}) + (\text{Ops} \times 2)$	$\$5,000 + (\$10,000 \times 2) = \$25,000$
Cost per TB stored (2 years)	$(\text{Total cost}) \div (36.5 \text{ TB stored})$	$\$25,000 \div 36.5 \text{ TB} \approx \$685 \text{ per TB (over 2 years)}$

Trade-off Question:

If you reduce RF from 3 to 2, you halve storage costs but lose rack-fault tolerance. Is this acceptable? Justify.

Not recommended for Medi Queue AI's production queue/appointment data. Dropping RF from 3 to 2 halves storage cost, but with only 2 copies there is no safe way to place both replicas so the system survives a rack failure AND a simultaneous single-node failure elsewhere — losing one rack can leave some blocks with zero surviving copies if both replicas happened to sit in that rack, or leaves the system running at RF=1 (no redundancy) for any block that does survive. Because this data includes live queue state and no-show history that hospital staff actively rely on for same-day scheduling, an outage or data loss directly disrupts patient care, not just an analytics report. RF=3 should stay for the production cluster; RF=2 is acceptable only for a non-critical dev/test copy of the data, or for older archived logs (e.g. beyond 1 year) that could instead move to a cheaper cold-storage/erasure-coded tier rather than simply cutting replication on the live tier.

Part C: Mini-Problems

Mini-Problem 1: Scaling for Peak Traffic

In Month 6, Medi Queue AI onboards 50 new hospital branches (150 total), and richer clinical/queue telemetry (vitals capture, longer visit notes) doubles log generation to 10GB/branch/day.

New annual raw data: $150 \text{ branches} \times 10\text{GB/day} \times 365 \text{ days} = 547.5 \text{ TB/year}$ (at 2-year scale: 1095 TB)

Your task: Calculate new DataNode count, storage costs, and replication strategy.

New 2-year raw data = 1,095 TB. Total raw disk at $\text{RF}=3 = 1,095 \times 3 = 3,285 \text{ TB}$. Metadata overhead ($\sim 5\%$) = 164.25 TB. Usable capacity needed $\approx 3,285 + 164.25 = 3,449.25 \text{ TB}$. New DataNode count = $3,449.25 \text{ TB} \div 12 \text{ TB/node} \approx 288$ DataNodes (up from 10 — roughly a $29\times$ scale-up), which at 5 nodes/rack means about 58 racks. Cost: hardware = $288 \times \$500 = \$144,000$; annual ops = $288 \times \$1,000 = \$288,000/\text{year}$; 2-year total $\approx \$144,000 + (\$288,000 \times 2) = \$720,000$. Replication strategy: keep $\text{RF}=3$ for the live/recent tier given the same patient-care-critical reasoning as the main scenario; to control the steep cost growth, move logs older than, say, 6–12 months into a cheaper cold-storage or erasure-coded tier instead of scaling the full- $\text{RF}=3$ hot tier linearly forever.

Mini-Problem 2: Block Size Tuning

Experiment: What if we use 256MB blocks instead of 128MB?

Implications:

- Fewer blocks $\rightarrow$ fewer map tasks $\rightarrow$ less parallelism
- Fewer blocks $\rightarrow$ less NameNode memory

Estimate: With 256MB blocks, how many mappers for the 36.5TB dataset? Pros and cons?

With 256MB blocks: $36.5 \text{ TB} \div 256 \text{ MB} \approx 146,000 \text{ blocks} \rightarrow \approx 146,000 \text{ mappers}$ (roughly half of the 128MB case). Pros: half as many blocks means half the NameNode metadata/heap pressure, and fewer, larger map tasks means less per-task scheduling overhead — good for the large nightly batch job that retrains the no-show prediction model over the full historical dataset. Cons: fewer, larger tasks reduce parallelism, so each mapper takes longer and the job may run slower overall if the cluster has spare capacity that now goes unused; a single mapper failure also wastes more completed work before it is retried. Decision: 256MB is a reasonable choice for this workload, since the job is a large, latency-tolerant nightly batch run rather than a low-latency interactive query.

Mini-Problem 3: Failure Recovery Timeline

A DataNode in Rack 1 crashes (hardware failure). HDFS detects missing replicas and re-replicates the blocks to other DataNodes. Estimate the re-replication time:

- DataNode disk: 12 TB
- Network bandwidth (typical): 1 Gbps

How long does it take to re-replicate 12 TB from peer DataNodes?

1 Gbps $\approx 125 \text{ MB/s}$ of usable throughput. $12 \text{ TB} \approx 12,582,912 \text{ MB}$. Time = $12,582,912 \text{ MB} \div 125 \text{ MB/s} \approx 100,663 \text{ seconds} \approx 28 \text{ hours}$ if re-replicated over a single 1Gbps link. In practice HDFS re-replicates from and to multiple peer DataNodes in parallel, so real recovery is typically much faster than this single-link worst case (e.g. spreading the copy across 5–6 peers in parallel could bring it down to roughly 5–6 hours). Even so, a ~ 1 -day worst-case recovery window is a meaningful operational risk for a hospital queue system, and argues for either upgrading inter-node bandwidth (e.g. 10Gbps) or keeping DataNode disk sizes smaller so any single failure has less data to re-replicate.

Mini-Problem 4: Compression Trade-offs

Compress the logs with Gzip (typical compression ratio 3:1 for text logs). How does this affect storage and MapReduce job performance?

New compressed raw data: $36.5 \text{ TB} \div 3 = 12.17 \text{ TB}$

- Pro: Storage cost drops by $2/3$
- Con: Mappers must decompress on-the-fly (higher CPU) Decision: Should we compress? Justify based on the numbers.

Compressed raw data = $36.5 \text{ TB} \div 3 \approx 12.17 \text{ TB}$, so total disk at RF=3 drops from 109.5 TB to about 36.5 TB — roughly a 66% storage cut, meaning the cluster could shrink from ~10 DataNodes to about 4, a large cost saving. However, plain Gzip is not splittable: a single large Gzip file must be processed start-to-finish by one mapper, which would badly hurt the parallelism this cluster was sized for. Decision: Yes, compress — but use a splittable codec (e.g. Snappy or bzip2) or keep individual compressed files close to the block size (~128MB) rather than one giant Gzip file per branch, so the storage savings are captured without sacrificing MapReduce parallelism for the nightly no-show analytics job.

Part D: Problem Bank – 4 Additional Scenarios

Scenario A: Real-Time Data Warehouse (E-commerce)

An e-commerce platform collects real-time user behavior (click streams, purchases) from mobile and web. Volume: 2TB/day. Need to query data within 1 hour for BI dashboards. Design an HDFS cluster with constraints: (1) RF=2 for cost, (2) 30-node cluster budget, (3) 1-day data retention (rolling 24-hour window). Deliverables: Cluster sizing, replication strategy, block size justification, 24-hour re-replication risk.

Cluster sizing: steady-state volume is only ~2TB (rolling 24-hour window); at RF=2 that's 4TB raw plus ~5% overhead ≈ 4.2TB usable — trivially small versus a 30-node budget, so all 30 nodes are used mainly for I/O parallelism and concurrent BI-query throughput rather than raw capacity. Replication strategy: RF=2 as mandated; since there is no third “spare” copy, both replicas should be placed on different racks (rather than the usual 2-same-rack/1-cross-rack pattern) so a single rack failure still leaves one live copy. Block size: smaller blocks (64–128MB) to maximize parallel mappers, since dashboards must reflect data within 1 hour and more, smaller tasks finish the job faster. 24-hour re-replication risk: because data ages out after 24 hours anyway, the exposure window is inherently short-lived, but if a node fails, blocks briefly sit at RF=1 with no spare copy until re-replication completes — fast (e.g. 10Gbps) inter-rack bandwidth is important here to close that gap before the 1-hour BI SLA is affected.

Scenario B: Backup & Archive (Regulatory Compliance)

A financial services firm must retain transaction logs for 7 years (compliance). Volume: 500GB/day. Budget: minimize cost (data is infrequently accessed). Design an HDFS cluster with: (1) RF=1 for cost, (2) separate cold-storage tier for logs older than 1 year, (3) quarterly backups to off-site. Deliverables: Cost analysis (7-year total), replication strategy, backup plan, data recovery RTO/RPO.

7-year volume: $500\text{GB/day} \times 365 \times 7 \approx 1,277.5\text{ TB raw}$. At RF=1 (as specified): raw disk needed ≈ 1,277.5 TB, needing ≈ 107 DataNodes at 12TB/node. Cost: hardware ≈ $107 \times \$500 = \$53,500$; annual ops ≈ $107 \times \$1,000 = \$107,000/\text{year} \times 7\text{ years} = \$749,000$; 7-year total ≈ \$802,500. Replication strategy: RF=1 alone is a single point of failure per block, which is unacceptable for regulated financial data on its own — pair it with a cheaper cold-storage tier (e.g. erasure coding or RF=2) for logs older than 1 year to add some redundancy without paying full RF=3 cost. Backup plan: quarterly full backups to an off-site location (tape or object storage) as the real safety net, since HDFS RF=1 is not durable on its own. RTO/RPO: with only quarterly backups, RPO is as high as ~3 months of data at risk between backups — recommend at least weekly incremental backups given the compliance stakes; RTO depends on the restore path but should be budgeted at hours for recent hot data and potentially over a day for a full historical restore.

Scenario C: IoT Sensor Network (Streaming + Batch)

IoT devices (temperature, humidity, pressure) stream data from 10,000 sensors at 1 reading/min (14.4M readings/day). Data volume: ~2GB/day (small payloads, high frequency). Need to: (1) retain 2 years, (2) detect anomalies in real-time (need fast access), (3) run weekly batch analytics. Deliverables: HDFS cluster design (NameNode heap for millions of small files!), block size & replica count, comparison with time-series DB (InfluxDB, etc.).

$10,000\text{ sensors} \times 1\text{ reading/min} = 14.4\text{M readings/day}$. Storing one file per reading would create ~14.4M files/day $\times 730\text{ days} \approx 10.5\text{ billion files over 2 years}$ — at 1GB heap per 1M files that is ≈ 10.5TB of NameNode heap alone, which is completely impractical (the classic HDFS small-file problem). Fix: batch readings into larger files, e.g. one merged file per hour across all sensors → $24 \times 730 \approx 17,520\text{ files over 2 years}$, which needs negligible NameNode heap. Block size & replica count: total volume is small (~2GB/day ≈ 1.46TB over 2 years), so the default 128MB block size (or even smaller, e.g. 64MB) is fine; RF=3 is recommended so anomaly-relevant readings are never lost. Comparison with a time-series DB: HDFS/MapReduce is a poor fit for the “1 reading/min per sensor, detect anomalies in real time” requirement — a dedicated time-series database (InfluxDB, TimescaleDB) offers indexed, sub-second time-range queries and built-in downsampling/retention that HDFS lacks. Recommended design: use the time-series DB for the real-time/hot path and anomaly alerts, and periodically export/roll up data into HDFS purely for the weekly batch analytics and 2-year cold retention.

Scenario D: Multi-Tenant Cloud (SaaS)

A SaaS platform provides data analytics to multiple customers. Each customer's data is isolated in separate HDFS directories. Total: 100 customers, 1TB/customer on average. Constraint: (1) RF=3 for HA, (2) each customer must not access another's data. Deliverables: Cluster architecture (single cluster + multi-tenancy or separate clusters?), HDFS permissions & quotas, cost-per-customer estimate, cross-tenant security risks.

Criteria	Wt.	L4 (40–50)	L3 (30–39)	L2 (20–29)	L1 (0–19)
HDFS Concepts Understanding	20%	Accurate understanding of blocks, replication, NameNode role	Mostly correct; minor gaps	Basic understanding; some errors	Misinterprets core concepts
Constraint Analysis	20%	Identifies all constraints (cost, FT, scale); prioritizes well	Most constraints identified; minor prioritization gaps	Some constraints missed or poorly weighted	Ignores constraints to analyze
Design & Calculations	30%	Correct cluster sizing, RF strategy, topology; justified	Mostly correct; minor arithmetic/logic gaps	Correct approach but calculation errors	Incorrect design or major elements
Trade-off & Risk Analysis	20%	Thorough analysis: cost vs. availability; risk mitigation	Good analysis; some trade-offs missed	Basic analysis; limited depth	Little or no trade-off analysis
Documentation & Clarity	10%	Clear diagrams, formulas, step-by-step reasoning	Mostly clear; minor formatting issues	Understandable but lacks detail	Unclear or poorly organized
Workload size: 100 customers $\times$ 1TB average = 100TB raw; at RF=3 that's 300TB + ~5% overhead $\approx$ 315TB, needing $\approx$ 27–28 DataNodes at 12TB/node. Architecture: a single shared HDFS cluster with one directory per tenant (e.g. /tenants/<customer_id>/) is more cost-efficient than a separate cluster per customer, since most customers are far below the average and dedicated clusters would waste idle capacity; only unusually large or heavily regulated customers would justify carving out a dedicated cluster. HDFS permissions & quotas: give each tenant directory its own owning service account with restrictive (e.g. 700) permissions, plus per-directory HDFS space quotas and file-count quotas so no single tenant can consume the whole cluster's capacity or NameNode file budget. Cost per customer: roughly $(28 \text{ nodes} \times \\$500 \text{ hardware} + 28 \times \\$1,000 \text{ annual ops}) \div 100 \text{ customers} \approx \\420 per customer in the first year (hardware amortized once, ops recurring). Cross-tenant security risks: misconfigured directory permissions leaking one tenant's data to another, a compromised shared NameNode or edge node exposing all tenants at once (a large blast radius from shared infrastructure), noisy-neighbour resource contention where one tenant's heavy MapReduce job degrades others' performance, and metadata-level leakage (e.g. directory listings revealing other tenants' existence) if top-level permissions are not locked down.					

Evaluation Rubric

Total Marks: 100 (20 + 20 + 30 + 20 + 10)

CSA15 CO5 AT1 | Constraint-Based HDFS Design Workbook | Dr. C. Rajesh Babu